

Desarrollo en React JS

JavaScript avanzado

Centro de
e-Learning



UTN.BA

UNIVERSIDAD TECNOLÓGICA NACIONAL
CENTRO DE E-LEARNING

Presentación

En esta unidad profundizamos en conceptos clave para desarrollar aplicaciones modernas en JavaScript: programación orientada a objetos, métodos avanzados de arrays, asincronía, promesas, y el uso de funciones asincrónicas.

Estos conocimientos son fundamentales para entender cómo trabajan internamente muchas bibliotecas como React.

Objetivos de esta unidad

- **Comprender** qué es la programación orientada a objetos en JS.
- **Utilizar** clases, constructores y métodos.
- **Aplicar** métodos de array como map, filter, reduce.
- **Comprender** la asincronía y su necesidad en la web.
- **Manejar** promesas, async/await y errores de forma eficaz.
- **Aplicar** buenas prácticas en el manejo del flujo de datos.

¿Qué es la programación orientada a objetos (POO)?

La POO permite organizar el código en torno a entidades que combinan datos (propiedades) y comportamientos (métodos).

En JS, todo puede representarse como un objeto.

Este paradigma mejora la reutilización, escalabilidad y organización del código.

Clases en Javascript

Desde ES6 se puede usar la palabra clave `class` para definir clases.

Un constructor inicializa los valores, y los métodos se definen dentro de la clase.

Ejemplo:



```
class Animal {  
  constructor(nombre) {  
    this.nombre = nombre;  
  }  
  hablar() {  
    console.log(`${this.nombre} hace un sonido`);  
  }  
}
```

Herencia en JS

JavaScript permite herencia mediante **extends**.

Una subclase puede acceder y sobrescribir métodos de la clase padre.

Se usa **super()** para llamar al constructor de la clase superior.

Ejemplo:



```
class Perro extends Animal {  
  hablar() {  
    console.log(`${this.nombre} dice guau`);  
  }  
}
```

Métodos avanzados de arrays

JavaScript ofrece métodos que permiten trabajar de forma declarativa con arrays:

- **forEach()**: recorre elementos.
- **map()**: transforma cada elemento y retorna un nuevo array.
- **filter()**: filtra elementos según condición.
- **reduce()**: acumula un valor a partir de los elementos.
- **find()**, **some()**, **every()**: búsqueda y validación.

Ejemplo práctico con arrays



```
const productos = [  
  { nombre: "Remera", precio: 3000 },  
  { nombre: "Pantalón", precio: 9000 },  
  { nombre: "Zapatos", precio: 20000 },  
];  
  
const caros = productos  
  .filter(p => p.precio > 5000)  
  .map(p => p.nombre.toUpperCase());
```


Sincronía vs. asincronía

JavaScript es single-threaded, ejecuta una línea a la vez.

Pero para no bloquear al usuario, ciertas operaciones (como peticiones HTTP) se manejan de forma asincrónica.

Se delegan al navegador y se gestionan con *callbacks*, promesas o *async/await*.

Temporizadores

- **setTimeout(fn, ms)**: ejecuta una función una vez después del tiempo indicado.
- **setInterval(fn, ms)**: ejecuta la función repetidamente cada intervalo de tiempo.

Se usan para retrasar o repetir acciones, como animaciones o recargas periódicas.

Introducción a las promesas

Una promesa representa un proceso asíncronico.

Puede estar en estado pendiente, cumplida o rechazada.

Se usan métodos **.then()** y **.catch()** para manejar el resultado o errores.

Ejemplo:



```
fetch("https://api.com/datos")  
  .then(res => res.json())  
  .then(data => console.log(data))  
  .catch(error => console.error(error));
```

Crear promesas personalizadas

Se pueden envolver tareas asincrónicas dentro de una promesa:



```
function esperar(ms) {  
  return new Promise(resolve => setTimeout(resolve, ms));  
}  
esperar(2000).then(() => console.log("Pasaron 2 segundos"));
```

async y await

- **async** marca una función que siempre devuelve una promesa.
- **await** pausa la ejecución hasta que la promesa se resuelva.

Ejemplo:



```
async function cargar() {  
  const res = await fetch("https://api.com");  
  const data = await res.json();  
  console.log(data);  
}
```

Manejo de errores en async/await

Cuando se usa `await`, se deben manejar errores con `try/catch`:



```
try {  
  const res = await fetch("url");  
  const data = await res.json();  
} catch (error) {  
  console.error("Error al cargar datos", error);  
}
```

Buenas prácticas en asincronía

Es importante no mezclar el uso de **.then** con **await**, ya que puede generar confusión y errores difíciles de rastrear.

También se debe **manejar correctamente todos los errores posibles** utilizando bloques **try/catch** para evitar comportamientos inesperados en la aplicación.

En cuanto a las operaciones asincrónicas, se recomienda **evitar el uso de múltiples await en paralelo** cuando no existe una dependencia entre ellas, ya que esto puede afectar el rendimiento.

Buenas prácticas en asincronía

Las funciones asincrónicas deben nombrarse con verbos claros que indiquen su propósito, como **cargarDatos** u **obtenerUsuario**, para facilitar la comprensión del código.

Finalmente, es buena práctica **reutilizar la lógica asincrónica** mediante funciones bien diseñadas y organizadas, lo que mejora la mantenibilidad del proyecto.

¡Gracias!

**Centro de
e-Learning**



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL DE BUENOS AIRES